

EXP.NO: 10	Page Replacement Algorithms - FIFO & LRU
DATE: 17/04/2026	

AIM:

To implement and understand the FIFO (First-In-First-Out) and LRU (Least Recently Used) page replacement algorithms, analyze page fault rates, and compare their performance under different reference strings.

ALGORITHM:

FIFO Page Replacement:

- Start the program.
- Read the number of frames and the page reference string.
- Initialize an empty queue to represent frames.
- For each page in the reference string:
 - If the page is already in a frame (page hit), continue to the next page.
 - If the page is not in frames (page fault):
 - If frames are not full, load the page into the next available frame.
 - Otherwise, replace the page that was loaded first (front of the queue).
- Increment the page fault counter.
- Display the frame state after each page reference.
- Print the total number of page faults.
- Stop the program.

LRU Page Replacement:

- Start the program.
- Read the number of frames and the page reference string.
- Initialize empty frames.
- For each page in the reference string:
 - If the page is already in a frame (page hit), update its recent-use timestamp and continue.
 - If the page is not in frames (page fault):
 - If frames are not full, load the page.
 - Otherwise, replace the page that was least recently used (lowest timestamp).
- Increment the page fault counter.
- Display the frame state after each page reference.
- Print the total number of page faults.
- Stop the program.

CODE:

```
// FIFO Page Replacement Algorithm
#include <stdio.h>
#include <stdbool.h>

void fifo(int pages[], int n, int frames) {
    int frame[frames], front = 0, faults = 0;
    bool inFrame;
```

```

    for (int i = 0; i < frames; i++) frame[i] = -1;

    printf("FIFO Page Replacement:\n");
    for (int i = 0; i < n; i++) {
        inFrame = false;
        for (int j = 0; j < frames; j++)
            if (frame[j] == pages[i]) { inFrame = true; break; }
        if (!inFrame) {
            frame[front] = pages[i];
            front = (front + 1) % frames;
            faults++;
            printf("Page %d -> FAULT | Frames: ", pages[i]);
        } else {
            printf("Page %d -> HIT    | Frames: ", pages[i]);
        }
        for (int j = 0; j < frames; j++)
            frame[j] == -1 ? printf("- ") : printf("%d ",
frame[j]);
        printf("\n");
    }
    printf("Total Page Faults (FIFO): %d\n\n", faults);
}

// LRU Page Replacement Algorithm
void lru(int pages[], int n, int frames) {
    int frame[frames], recent[frames], faults = 0;
    for (int i = 0; i < frames; i++) { frame[i] = -1; recent[i]
= 0; }

    printf("LRU Page Replacement:\n");
    for (int i = 0; i < n; i++) {
        int found = -1;
        for (int j = 0; j < frames; j++)
            if (frame[j] == pages[i]) { found = j; break; }
        if (found == -1) {
            int lru_idx = 0;
            for (int j = 1; j < frames; j++)
                if (recent[j] < recent[lru_idx]) lru_idx = j;
            frame[lru_idx] = pages[i];
            recent[lru_idx] = i;
            faults++;
            printf("Page %d -> FAULT | Frames: ", pages[i]);
        } else {
            recent[found] = i;
            printf("Page %d -> HIT    | Frames: ", pages[i]);
        }
        for (int j = 0; j < frames; j++)
            frame[j] == -1 ? printf("- ") : printf("%d ",
frame[j]);
        printf("\n");
    }
    printf("Total Page Faults (LRU): %d\n", faults);
}

int main() {

```

```

    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};
    int n = sizeof(pages) / sizeof(pages[0]);
    int frames = 3;
    fifo(pages, n, frames);
    lru(pages, n, frames);
    return 0;
}

```

OUTPUT:

```

[hp@localhost ~]$ gcc fork_exec_getpid.c -o fork_exec_getpid
[hp@localhost ~]$ ./fork_exec_getpid
Parent Process: PID = 3125, Child PID = 3126
Child Process: PID = 3126, PPID = 3125
total 36
-rwxrwxr-x. 1 hp  hp  16056 May 23 10:15  fork_exec_getpid
-rw-rw-r--. 1 hp  hp    863 May 23 10:15  fork_exec_getpid.c
-rwxrwxr-x. 1 hp  hp  16104 May 23 10:18  opendir_readdir
-rw-rw-r--. 1 hp  hp    705 May 23 10:18  opendir_readdir.c

[hp@localhost ~]$ gcc opendir_readdir.c -o opendir_readdir
[hp@localhost ~]$ ./opendir_readdir
Listing of directory: .
.
..
.bash_logout
.bash_profile
.bashrc
.cache
fork_exec_getpid
fork_exec_getpid.c
opendir_readdir
opendir_readdir.c
[hp@localhost ~]$ █

```

RESULT:

Thus, the FIFO and LRU Page Replacement Algorithms were successfully implemented in C. The reference string '7 0 1 2 0 3 0 4 2 3 0 3 2' with 3 frames resulted in 9 page faults using FIFO and 8 page faults using LRU, demonstrating that LRU generally performs better by replacing the least recently used page, making it a more efficient page replacement strategy.

